How HRR Learns

Storing, gradients, and the comparison with learned bilinear maps and attention

James Pustejovsky

Brandeis University
Department of Computer Science

CS 135: Computational Semantics – Companion to the HRR session

Fall 2026

Roadmap

I	The short answer	what learning is in HRR, against a transformer
II	Learning by storing	one-shot superposition; the Hebbian rule, folded; forgetting; in-context memory
III	Every binding is a bilinear map	fixed, learned, and learned in low rank
IV	Learning by gradients	the gradient of a binding is an unbinding; the unitary fix; Plate's recurrent networks
V	Learned bilinear maps for relations	what a directional example writes; an experiment; ComplEx
VI	Attention as a learned binding	the learned form; softmax as cleanup; positions; two timescales
VII	Close	the summary comparison; the research question; exercises; reading

The HRR session's system learned nothing: random atoms, fixed operations, a stored list. Today, where learning can enter it, and how that compares with the two learned systems this course has met, bilinear forms and attention.

Where We Are

The HRR session, in five lines. Dimension n; atoms have variance $1/n$:

operation	**definition**	**status**
atoms	random vectors: length about 1, nearly orthogonal	drawn once, never changed
bind	circular convolution, $(\mathbf{a} \circledast \mathbf{b})_j = \sum_k a_k \, b_{(j-k) \bmod n}$	fixed
unbind	correlation, $(\mathbf{a} \# \mathbf{b})_j = \sum_k a_k \, b_{(k+j) \bmod n} = (\mathbf{a}^* \circledast \mathbf{b})_j$	fixed
bundle	addition	fixed
cleanup	return the stored atom with the highest cosine	a stored list

Nothing in this list is trained. Every operation is differentiable, though, and a trace grows by plain addition: the two routes by which learning enters.

Session 11c: convolution is the fold of the outer product, $\mathrm{fold}(\mathbf{u}\mathbf{v}^\top) = \mathbf{u} \circledast \mathbf{v} = (4, 5, 2, 5)$ for $\mathbf{u} = (1, 2, 0, 1)$, $\mathbf{v} = (2, 0, 1, 1)$; 11c's output index k is written j here, as in the HRR session. The involution $\mathbf{a}^*$ keeps a_0 and reverses the rest. The bonus deck writes correlation as $\star$; here it is Plate's $\#$.

What Learning Is in HRR

In classic HRR, learning is storing. A new fact is one addition to the trace, $\mathbf{t} \leftarrow \mathbf{t} + \mathbf{k} \circledast \mathbf{v}$, and recall is unbinding plus cleanup, $\mathbf{k} \# \mathbf{t} \approx \mathbf{v}$. Nothing else changes: the atoms were drawn once, the operations are fixed, no error is computed. A transformer learns the other way:

	HRR	**transformer (attention)**
what changes	one trace of n numbers	the weights: embeddings, W_Q, W_K, W_V
how	add a binding; no error signal	gradient descent on a prediction error
examples needed	one per fact	many, over many passes
where a fact ends up	in the trace, read by $\mathbf{k} \# \mathbf{t}$	spread over the weights; no one place to read it
cost of more facts	each adds noise to every other	later training can overwrite earlier learning
one-shot learning	the trace itself	the context, as stored keys and values

The two meet in the middle: HRR's atoms can be trained, and the gradient of a binding is an unbinding (Part IV); attention's score is a learned bilinear form with a soft cleanup (Part VI).

Part II
Learning by Storing

Storing a Fact Is Adding a Binding

HRR's memory is a **trace**: one vector $\mathbf{t}$ of n numbers, not a matrix. It learns a new fact, a key $\mathbf{k}$ bound to a value $\mathbf{v}$, in one step, $\mathbf{t} \leftarrow \mathbf{t} + \mathbf{k} \circledast \mathbf{v}$: no gradient, no pass over old data. Irregular plurals, one at a time, singular as key and plural as value:

$$\mathbf{t} = \mathbf{mouse} \circledast \mathbf{mice} + \mathbf{goose} \circledast \mathbf{geese} + \mathbf{child} \circledast \mathbf{children}$$

Learning *ox, oxen* is one more addition, $\mathbf{t} \leftarrow \mathbf{t} + \mathbf{ox} \circledast \mathbf{oxen}$. Query by unbinding, $\mathbf{goose} \# \mathbf{t} \approx \mathbf{geese} +$ noise, then clean up against the word list.

The price of one-shot learning is interference. With k pairs stored (a count, not the key $\mathbf{k}$), every recall scores about $1/\sqrt{k+1}$:

pairs stored, k	1	5	20	50
recall cosine, $1/\sqrt{k+1}$	0.71	0.41	0.22	0.14
cleanup among 1,000 words correct, $n = 1{,}024$	100%	100%	100%	85%

Cosines from the formula, which the HRR session's measurements matched to two decimals up to $k = 20$; cleanup figures from its capacity table.

Learning is instant, and every fact learned makes every other fact a little noisier.

The Same Rule, Unfolded

index	0	1	2	3
u	1	2	0	1
v	2	0	1	1

The classic associative memory stores the outer product, $W \leftarrow W + \mathbf{v}\mathbf{k}^\top$: a **Hebbian**, or correlation-matrix, memory, read by $W\mathbf{k}_j = \mathbf{v}_j\,(\mathbf{k}_j \cdot \mathbf{k}_j) + \text{crosstalk}$. HRR stores the fold of the same outer product, since $\text{fold}(\mathbf{k}\mathbf{v}^\top) = \mathbf{k} \circledast \mathbf{v}$ (Session 11c). By hand:

$$\mathbf{u}\mathbf{v}^\top = \begin{pmatrix} 2 & 0 & 1 & 1 \\ 4 & 0 & 2 & 2 \\ 0 & 0 & 0 & 0 \\ 2 & 0 & 1 & 1 \end{pmatrix} \xrightarrow{\text{fold}} \mathbf{u} \circledast \mathbf{v} = (4, 5, 2, 5) : \qquad 16 \text{ numbers become } 4$$

	outer-product memory	**HRR trace**
numbers	n^2	n
recall with k pairs	exact, for up to n orthonormal keys	about $1/\sqrt{k+1}$
pairs cleanup can read	up to n	about $n/(2 \ln m)$

The learning rule is the same; HRR compresses what it stores. (m: candidates in cleanup.)

Forgetting by Decay

Shrink the trace before each addition, $\mathbf{t} \leftarrow \lambda \mathbf{t} + \mathbf{k} \circledast \mathbf{v}$ with $\lambda < 1$. Old facts fade, and new ones are recalled better. Store 50 facts, then recall each by its age, the number of facts stored after it:

age of the fact	0	1	5	10	20	49
$\lambda = 1$	0.14	0.15	0.14	0.14	0.14	0.14
$\lambda = 0.95$	0.29	0.29	0.24	0.19	0.11	0.02
$\lambda = 0.90$	0.40	0.37	0.26	0.15	0.06	0.00

Recall cosine, $n = 1{,}024$, mean of 100 runs.

With $\lambda = 1$ every fact sits at $1/\sqrt{51} \approx 0.14$, recent or old. With $\lambda = 0.9$ the newest is at 0.40 and the oldest is gone: a fact's signal shrinks by λ with each later fact, and so does its share of the noise.

Decay buys recency with the past. Fast-weight memories use the same knob: Ba et al. (2016) update $A \leftarrow \lambda A + \eta\, \mathbf{h}\mathbf{h}^\top$, an outer-product memory with decay.

Room, Not the Rule

Could a smarter storing rule beat $1/\sqrt{k+1}$? The **delta rule** stores only what the memory still gets wrong. No rule can beat the *least-squares* trace, the n numbers that make recall error smallest. Compare:

memory $(n = 256)$	**pairs**	**plain addition**	**best rule**
HRR trace, random keys	$k = 5$	0.40	0.44 least squares
	$k = 20$	0.220	0.225 least squares
HRR trace, unitary keys	$k = 5,\ 20$	$0.45,\ 0.22$	the same
outer-product memory, n^2 numbers	$k = 40$	0.93	1.00 delta rule
	$k = 200$	0.75	0.99 delta rule

Mean recall cosine over 20–40 runs.

Schlag, Irie & Schmidhuber (2021) give linear attention a delta rule, and it pays: n^2 numbers leave room for many exact pairs. An HRR trace is limited by its n numbers, not by its learning rule. With unitary keys, least squares only rescales plain addition.

In-Context Learning, Compared

A transformer learns from its context without changing a weight: each token's key and value are stored, and later queries read them. Three ways to store k pairs:

	stores	**numbers**	**exact?**	**a new pair**
KV cache (softmax attention)	every $\mathbf{k}_i$, $\mathbf{v}_i$	$2nk$, growing	yes	append it
fast weights (linear attention)	$\sum_i \mathbf{v}_i \mathbf{k}_i^\top$	n^2 per head	up to n pairs†	add $\mathbf{v}\mathbf{k}^\top$
HRR trace	$\sum_i \mathbf{k}_i \circledast \mathbf{v}_i$	n	no: $1/\sqrt{k+1}$	add $\mathbf{k} \circledast \mathbf{v}$

†With the delta rule of the last slide.

Schlag et al. (2021) show that linear attention *is* a fast-weight memory (Schmidhuber 1992): a slow network learns by gradient descent to write outer products of keys and values into a fast one.

An HRR trace is that memory, folded. The bonus deck follows Hrrformer (Alam et al. 2023), which folds it inside attention: n numbers per head instead of n^2, paid for in crosstalk.

In-Class Exercise 1

index	0	1	2	3
$\mathbf{u}$	1	2	0	1
$\mathbf{v}$	2	0	1	1

1. Store $\mathbf{u} \circledast \mathbf{v}$ in an empty trace, then store the reversed pair $\mathbf{v} \circledast \mathbf{u}$. What does the trace hold, and what does that say about learning a direction by storing?
2. With decay $\lambda = 0.9$, by what factor has a fact's signal shrunk after ten more facts are stored?
3. Why does the least-squares trace equal plain addition, up to scale, when every key is unitary?
4. $\star$ An outer-product memory at $n = 256$ uses $65{,}536$ numbers and holds 200 pairs at 0.99. An HRR trace with $n = 65{,}536$ uses the same budget. About how many pairs can cleanup read from it, with $m = 1{,}000$ candidates?

Seven minutes. Item 4 is the starred one.

In-Class Exercise 1: Answers

1. $(4,5,2,5)+(4,5,2,5)=(8,10,4,10)$: since $\mathbf{v}\circledast\mathbf{u}=\mathbf{u}\circledast\mathbf{v}$, storing the reversed pair is storing the original twice. A trace of bindings cannot learn a direction, however many examples it sees; the direction has to come from the roles.

2. $0.9^{10}\approx 0.35$. Its share of the noise shrinks too, which is why the measured recall falls from 0.40 to 0.15, a factor of about 0.38.

3. Least squares works frequency by frequency and divides by $\sum_i |\mathcal{F}_f\mathbf{k}_i|^2$. For unitary keys every $|\mathcal{F}_f\mathbf{k}_i|=1$, so the divisor is k at every frequency: plain addition, divided by k. A constant factor does not change a cosine.

4. ⋆ About $n/(2\ln m)=65{,}536/13.8\approx 4{,}700$ pairs, each read noisily and cleaned up; the outer-product memory reads at most 256, each exactly. For the same numbers, HRR holds more pairs, and holds each one less precisely.

Part III
Every Binding Is a Bilinear Map

Outer Product, Then a Linear Map

Any binding that is linear in each argument has the form $\mathbf{z} = W(\mathbf{x}\mathbf{y}^\top)$: build the outer product, then apply a linear map W. The systems of this course differ only in W:

system	**the map W**	**output**	**learned numbers**
tensor product (TPR)	the identity: keep all of $\mathbf{x}\mathbf{y}^\top$	n^2 numbers	none
HRR	the fold	n numbers	none
a learned bilinear layer	any linear map to m outputs	m numbers	$m\,n^2$
a bilinear form, $\mathbf{x}^\top G\,\mathbf{y}$	the sum $\langle G, \mathbf{x}\mathbf{y}^\top \rangle$	1 number	n^2
an attention head	the same, with $G = W_Q^\top W_K$	1 number	$2\,d_k\,n$

TPR keeps everything, HRR fixes the map, and learned models learn it: in full (a bilinear layer, RESCAL) or in low rank (an attention head, whose G has rank at most d_k).

$\langle G, M \rangle = \sum_{ij} G_{ij} M_{ij}$, so $\mathbf{x}^\top G\,\mathbf{y} = \langle G, \mathbf{x}\mathbf{y}^\top \rangle$: a learned bilinear form is a learned linear readout of the outer product.

Where Order Can Live

Split the outer product, $\mathbf{x}\mathbf{y}^\top = S + A$, as in Session 11c. All of the order is in A, which changes sign when $\mathbf{x}$ and $\mathbf{y}$ swap. Which maps can see it?

map	**what A contributes**	**order is**
the fold (HRR binding)	0: the fold annihilates A (Session 11c)	invisible
a diagonal form (DistMult)	$\sum_i r_i A_{ii} = 0$: A has a zero diagonal	invisible
correlation (HolE)	the odd part of the output (bonus deck)	visible
a full form G (RESCAL)	$\langle G, A\rangle = \langle G_A, A\rangle$	visible, through G_A
an attention head	the same, with $G = W_Q^\top W_K$	visible, through G_A
TPR	all of it, kept	visible

$G_A = \frac{1}{2}(G - G^\top)$ is the antisymmetric part of G; its symmetric part pairs with A to give 0.

Which half survives is fixed by the shape of the map before any learning starts. Learning can only fill in what the shape allows, as Part V will measure.

Part IV
Learning by Gradients

Gradients of a Bilinear Form

For a score $s = \mathbf{x}^\top G\,\mathbf{y}$, the three gradients are

$$\frac{\partial s}{\partial \mathbf{x}} = G\,\mathbf{y}, \qquad \frac{\partial s}{\partial \mathbf{y}} = G^\top \mathbf{x}, \qquad \frac{\partial s}{\partial G} = \mathbf{x}\mathbf{y}^\top$$

So a gradient step on a loss $L(s)$ changes G by $-\eta\, L'(s)\,\mathbf{x}\mathbf{y}^\top$: an outer product, weighted by how wrong the score was.

Gradient descent on a bilinear form is the Hebbian rule of Part II, applied where the model errs and as much as it errs.

An attention head factors its form, $s = (W_Q\mathbf{x}) \cdot (W_K\mathbf{y})$, and the outer products move into the factors:

$$\frac{\partial s}{\partial W_Q} = (W_K\mathbf{y})\,\mathbf{x}^\top, \qquad \frac{\partial s}{\partial W_K} = (W_Q\mathbf{x})\,\mathbf{y}^\top$$

Every learned bilinear model in this deck is trained by these formulas, or by the HRR versions on the next slides.

One Step From S to the Course's Head

The running example of Sessions 11 and 13: tokens $\mathbf{x}_1 = (3, 1)$ and $\mathbf{x}_2 = (1, 3)$. Start from a symmetric head, $G_0 = S = \left(\begin{smallmatrix} 1 & 1 \\ 1 & 1 \end{smallmatrix}\right)$, which scores 16 in both directions.

Teach it one direction: $\mathbf{x}_1$ should attend to $\mathbf{x}_2$, and $\mathbf{x}_2$ should not attend to $\mathbf{x}_1$. With the loss $L = s(\mathbf{x}_2, \mathbf{x}_1) - s(\mathbf{x}_1, \mathbf{x}_2)$, one step of size η adds

$$\eta \left(\mathbf{x}_1 \mathbf{x}_2^\top - \mathbf{x}_2 \mathbf{x}_1^\top \right) = \eta \left[\begin{pmatrix} 3 & 9 \\ 1 & 3 \end{pmatrix} - \begin{pmatrix} 3 & 1 \\ 9 & 3 \end{pmatrix} \right] = 8\eta \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}$$

With $\eta = 1/8$:

$$G_1 = \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix} = S + A, \qquad s(\mathbf{x}_1, \mathbf{x}_2) = 24 = 16 + 8, \qquad s(\mathbf{x}_2, \mathbf{x}_1) = 8 = 16 - 8$$

That is the course's head, one gradient step away from its symmetric half.

A directional example writes only into A: $\mathbf{x}_1 \mathbf{x}_2^\top - \mathbf{x}_2 \mathbf{x}_1^\top$ is antisymmetric for every $\mathbf{x}_1, \mathbf{x}_2$. An example that rewards both directions writes only into S.

The Gradient of a Binding Is an Unbinding

Now the HRR operations. For $\mathbf{y} = \mathbf{a} \circledast \mathbf{b}$, and a loss whose error signal is $\mathbf{g} = \partial L / \partial \mathbf{y}$:

$$\frac{\partial L}{\partial \mathbf{a}} = \mathbf{b} \# \mathbf{g}, \qquad \frac{\partial L}{\partial \mathbf{b}} = \mathbf{a} \# \mathbf{g}$$

Why. $y_j = \sum_k a_k \, b_{j-k}$, so $\dfrac{\partial L}{\partial a_k} = \sum_j g_j \, b_{j-k} = \sum_m b_m \, g_{m+k} = (\mathbf{b} \# \mathbf{g})_k$, with $m = j - k$.

Backpropagating through a binding unbinds the error by the other factor.

The same holds for HolE's score $s = \mathbf{r} \cdot (\mathbf{e}_s \# \mathbf{e}_o)$, which binds three vectors:

$$\frac{\partial s}{\partial \mathbf{r}} = \mathbf{e}_s \# \mathbf{e}_o, \qquad \frac{\partial s}{\partial \mathbf{e}_s} = \mathbf{r} \# \mathbf{e}_o, \qquad \frac{\partial s}{\partial \mathbf{e}_o} = \mathbf{r} \circledast \mathbf{e}_s$$

Each slot's gradient binds or unbinds the other two, and the gradient for the relation is the subject–object pair itself.

All five formulas checked against finite differences at $n = 8$.

Learning One Factor by Hand, Step 1

index	0	1	2	3
$\mathbf{u}$	1	2	0	1
$\mathbf{v}$	2	0	1	1

Suppose we know $\mathbf{v}$ and the bound pair $\mathbf{t} = (4, 5, 2, 5)$, and must learn the other factor $\mathbf{a}$, with $\mathbf{a} \circledast \mathbf{v} = \mathbf{t}$. The answer is $\mathbf{u}$. Use the loss $L = \frac{1}{2}\|\mathbf{a} \circledast \mathbf{v} - \mathbf{t}\|^2$, whose gradient is $\mathbf{v} \# (\mathbf{a} \circledast \mathbf{v} - \mathbf{t})$. Start at $\mathbf{a} = \mathbf{0}$. The error is $-\mathbf{t}$, so the gradient is $-(\mathbf{v} \# \mathbf{t})$: shift $\mathbf{t}$ back by j and dot with $\mathbf{v}$.

j	$\mathbf{t}$ shifted back by j, dotted with $\mathbf{v} = (2, 0, 1, 1)$	$(\mathbf{v} \# \mathbf{t})_j$
0	$(4, 5, 2, 5) \cdot \mathbf{v} = 8 + 0 + 2 + 5$	15
1	$(5, 2, 5, 4) \cdot \mathbf{v} = 10 + 0 + 5 + 4$	19
2	$(2, 5, 4, 5) \cdot \mathbf{v} = 4 + 0 + 4 + 5$	13
3	$(5, 4, 5, 2) \cdot \mathbf{v} = 10 + 0 + 5 + 2$	17

With step size $\eta = 1/16$: $\mathbf{a}_1 = (15, 19, 13, 17)/16 = (0.94,\ 1.19,\ 0.81,\ 1.06)$, at cosine 0.88 with $\mathbf{u}$.

The first gradient step is Plate's decoder: correlate the bound pair with the known factor. The HRR session computed exactly this vector, and the same 0.88.

Learning One Factor by Hand, Step After Step

index	0	1	2	3
$\mathbf{u}$	1	2	0	1
$\mathbf{v}$	2	0	1	1

Keep stepping. In the Fourier basis each frequency learns on its own: $\mathcal{F}(\mathbf{a} \circledast \mathbf{v}) = \mathcal{F}\mathbf{a} \odot \mathcal{F}\mathbf{v}$, so each step multiplies the error at frequency k by $1 - \eta\,|\mathcal{F}_k \mathbf{v}|^2$:

$$|\mathcal{F}\mathbf{v}|^2 = (16,\ 2,\ 4,\ 2) \qquad \Longrightarrow \qquad \text{factors } \left(0,\ \tfrac{7}{8},\ \tfrac{3}{4},\ \tfrac{7}{8}\right) \text{ at } \eta = \tfrac{1}{16}$$

step	0	1	2	3	5	10	20	40
loss	35	1.89	1.22	0.80	0.38	0.076	0.005	0.000
$\cos(\mathbf{a}, \mathbf{u})$	–	0.88	0.93	0.95	0.98	0.995	0.9997	1.000

Frequency 0 is learned in one step. Frequencies 1 and 3, where $\mathbf{v}$ is weakest, are learned last, losing an eighth of their error per step.

The limit is $\mathbf{a} = \mathbf{u} = \mathbf{t} \circledast \mathbf{v}^{-1}$: the exact inverse's answer, reached frequency by frequency.

First Step the Involution, Last Step the Exact Inverse

The pattern is general. Learn $\mathbf{a}$ from $\mathbf{a} = \mathbf{0}$ so that $\mathbf{a} \circledast \mathbf{b} \approx \mathbf{t}$:

	gives	**at frequency** k
first step	$\eta\,(\mathbf{b} \# \mathbf{t})$: the involution decoder	multiplies $\mathcal{F}_k\mathbf{t}$ by $\eta\,\overline{\mathcal{F}_k\mathbf{b}}$
the limit	$\mathbf{t} \circledast \mathbf{b}^{-1}$: the exact inverse	divides $\mathcal{F}_k\mathbf{t}$ by $\mathcal{F}_k\mathbf{b}$
in between	each frequency on its own clock	done once $\eta\,\lvert\mathcal{F}_k\mathbf{b}\rvert^2 \times$ steps $\gg 1$

So the weak frequencies are learned last: the same small coefficients that made the exact inverse unstable. For a random atom at $n = 1{,}024$, $|\mathcal{F}_k\mathbf{b}|$ runs from about 0.035 to 2.6 (medians over 300 atoms), so the weakest frequency needs about $5{,}000$ times as many steps as the strongest.

Stopping early keeps the decoder near the involution, which the HRR session found more robust on crowded traces: 0.58 against the exact inverse's 0.34, with two pairs.

The Unitary Fix, for Learning

Ganesan et al. (NeurIPS 2021) report that plain HRRs are numerically unstable inside gradient-trained networks. Their fix projects every vector to unit Fourier magnitudes:

$$\pi(\mathbf{x}) \;=\; \mathcal{F}^{-1}\!\left(\ldots,\; \frac{\mathcal{F}_k\mathbf{x}}{|\mathcal{F}_k\mathbf{x}|},\; \ldots\right)$$

That is the HRR session's **unitary** vector. In learning terms:

- the involution and the exact inverse coincide, so one pair decodes exactly;
- every frequency learns at the same rate: the $5{,}000$-to-1 spread of the last slide becomes 1;
- with $\eta = 1$, the first gradient step is already the answer.

They use projected HRRs for the output layer of extreme multi-label classification, with up to $670{,}000$ labels. The output layer shrinks by up to 99.55%; accuracy beats a standard layer on some datasets and falls below it on the largest.

The polar form shows the problem: if $\mathcal{F}_k\mathbf{a} = r_k e^{i\theta_k}$, the exact inverse has $\frac{1}{r_k}e^{-i\theta_k}$ and the involution $r_k e^{-i\theta_k}$. They agree exactly when every $r_k = 1$.

Plate's Holographic Recurrent Networks (1992)

Plate's own learning system: a recurrent network whose state is updated by convolution with a key, $\mathbf{h} \leftarrow \mathbf{p} \circledast \mathbf{h}$, and read out with the key's involution.

fixed	**learned**
the recurrence: convolution	the symbol vectors, as the output layer's weights
decoding: the involution	the code that starts each sequence, found by gradient search
	cleanup: a softmax over the learned symbol vectors

Reported. With 16 hidden units it produced about 90% of novel sequences up to length 12, ahead of a simple recurrent network trained on more data. Plate estimates over 1 bit per hidden unit, against about 0.1 bit per element for a memory of random vectors.

Learning the vectors made the same algebra roughly ten times more efficient.

Note the cleanup: a softmax over learned vectors, the shape attention takes in Part VI.

In-Class Exercise 2

index	0	1	2	3
$\mathbf{u}$	1	2	0	1
$\mathbf{v}$	2	0	1	1

1. Learn the other factor: with $\mathbf{a} = \mathbf{u}$ known and $\mathbf{t} = (4, 5, 2, 5)$, start from $\mathbf{b} = \mathbf{0}$ and take one step of size $\frac{1}{16}$ on $L = \frac{1}{2}\|\mathbf{u} \circledast \mathbf{b} - \mathbf{t}\|^2$. Which vector do you get, and how close is it to $\mathbf{v}$?
2. In the hand example, which frequency is learned in a single step at $\eta = \frac{1}{16}$, and why that one?
3. What is the largest step size for which learning $\mathbf{a}$ against $\mathbf{v}$ does not blow up?
4. $\star$ Explain why stopping gradient descent early acts like choosing the involution over the exact inverse.

Seven minutes. Item 4 is the starred one.

In-Class Exercise 2: Answers

1. The first step is $\frac{1}{16}(\mathbf{u} \# \mathbf{t}) = (19, 13, 17, 15)/16$, the HRR session's decoding of $\mathbf{v}$ from $\mathbf{u} \circledast \mathbf{v}$. Its cosine with $\mathbf{v}$ is 0.88.

2. Frequency 0: its factor is $1 - \frac{1}{16} \cdot 16 = 0$. The step size happens to equal $1/|\mathcal{F}_0 \mathbf{v}|^2$, so that frequency is solved at once.

3. Each factor $1 - \eta|\mathcal{F}_k \mathbf{v}|^2$ must stay between -1 and 1, so $\eta < 2/\max_k |\mathcal{F}_k \mathbf{v}|^2 = 2/16 = \frac{1}{8}$. The strongest frequency sets the speed limit; the weakest sets the time to converge.

4. $\star$ Early on, the weak frequencies have barely moved from their first-step values, which are the involution's; only later do they approach the exact inverse's divisions. Early stopping leaves the weak frequencies, where noise would be amplified, close to the involution: regularization, in the vocabulary of machine learning.

Part V
Learned Bilinear Maps for Relations

Four Ways to Score a Triple

A knowledge graph lists facts as triples: parent_of(Mary, John), married_to(Mary, Bill). Each model scores a triple with a bilinear form in the two entity vectors; they differ in the form's shape.

model	**score**(s, r, o)	**form**	**per relation**	**asymmetric?**
RESCAL (2011)	$\mathbf{e}_s^\top W_r \, \mathbf{e}_o$	any matrix	d^2	yes
DistMult (2015)	$\sum_i e_{s,i} \, r_i \, e_{o,i}$	diagonal	d	no
HolE (2016)	$\mathbf{r} \cdot (\mathbf{e}_s \# \mathbf{e}_o)$	circulant, $C(\mathbf{r})_{ij} = r_{j-i}$	d	yes
ComplEx (2016)	$\mathrm{Re} \sum_k r_k \, e_{s,k} \, \overline{e_{o,k}}$	complex diagonal	$2d$	yes

All four are trained the same way: a logistic loss pushes true triples up and corrupted ones down, by gradient descent. The shape decides what can be learned.

HolE's score is $\mathbf{e}_s^\top C(\mathbf{r}) \, \mathbf{e}_o$: correlation is a bilinear form whose matrix is circulant, each row a cyclic shift of $\mathbf{r}$. The bonus deck introduces RESCAL and HolE in detail.

What a Directional Example Writes

index	0	1	2	3
u	1	2	0	1
v	2	0	1	1

Train on (a, r, b) true and (b, r, a) false. The relation's update is $\propto \partial s(a, r, b) - \partial s(b, r, a)$:

model	**update for the relation**	**by hand, with $a = \mathbf{u}$, $b = \mathbf{v}$**
RESCAL	$\mathbf{a}\mathbf{b}^\top - \mathbf{b}\mathbf{a}^\top = 2A$	the matrix below
DistMult	$\mathbf{a} \odot \mathbf{b} - \mathbf{b} \odot \mathbf{a}$	$(0, 0, 0, 0)$
HolE	$\mathbf{a} \# \mathbf{b} - \mathbf{b} \# \mathbf{a} = 2\times$odd part	$(3, 4, 3, 6) - (3, 6, 3, 4) = (0, -2, 0, 2)$
convolution, $\mathbf{r} \cdot (\mathbf{a} \circledast \mathbf{b})$	$\mathbf{a} \circledast \mathbf{b} - \mathbf{b} \circledast \mathbf{a}$	$(0, 0, 0, 0)$

$$\mathbf{u}\mathbf{v}^\top - \mathbf{v}\mathbf{u}^\top = \begin{pmatrix} 0 & -4 & 1 & -1 \\ 4 & 0 & 2 & 2 \\ -1 & -2 & 0 & -1 \\ 1 & -2 & 1 & 0 \end{pmatrix}$$

the RESCAL update by hand: Session 11c's $2A$

DistMult's update is zero: no amount of data teaches it a direction. HolE writes direction into the odd part of $\mathbf{r}$, RESCAL into the antisymmetric part of W_r.

A model learns order only where its update can write it.

An Experiment: Family Relations

Four families of three generations: 44 people and 104 true triples, parent_of (48), sibling_of (48, both directions), married_to (8, both directions). Dimension $d = 32$; logistic loss with four random corruptions per true triple; Adam, $1{,}500$ passes; three runs.

	DistMult	**HolE**	**RESCAL**
numbers per relation	32	32	1,024
true triples scored true	99–100%	100%	100%
corrupted triples scored false	98–99%	100%	100%
reversed parent_of scored true	98–100%	0%	0%
reversed sibling_of, married_to scored true	100%	100%	100%

All three fit the data. DistMult also believes nearly every child is its own parent's parent, since its score cannot change when the arguments swap. HolE gets direction right with $1/32$ of RESCAL's numbers per relation.

Reversed sibling_of and married_to triples are true, so accepting them is correct.

What the Learned Relations Look Like

Split each learned relation into its symmetric and directional parts. For HolE the directional part is the odd part, $\frac{1}{2}(\mathbf{r} - \mathbf{r}^*)$; for RESCAL it is $\frac{1}{2}(W - W^\top)$.

relation	**HolE: odd share of $\|\mathbf{r}\|^2$**	**RESCAL: $\|A\|^2/\|W\|^2$**
parent_of	0.32–0.56	0.38–0.39
sibling_of	0.00	0.00
married_to	0.00	0.00–0.02

Ranges over the three runs of the last slide.

Learning wrote direction exactly where the relation has it: into the odd part of a HolE vector, and into the antisymmetric part of a RESCAL matrix. For the symmetric relations it wrote none, because their true triples come in both directions and their updates cancel, as the directional-example slide predicts.

Nobody told either model which relations are symmetric. The odd part was there to be used, and the data used it only for parent_of.

ComplEx Is HolE in the Fourier Basis

HolE's form is circulant, and a circulant is diagonal in the Fourier basis (the HRR session's Fourier view). For a real $\mathbf{r}$, its even part sets the real parts of $\mathcal{F}\mathbf{r}$, and its odd part the imaginary parts:

$$(0,-2,0,2) \xrightarrow{\mathcal{F}} (0,\ 4i,\ 0,\ -4i), \qquad (3,5,3,5) \xrightarrow{\mathcal{F}} (16,\ 0,\ -4,\ 0)$$

The first vector is the HolE update of the directional-example slide, the odd part of $\mathbf{u} \# \mathbf{v}$, doubled; the second is its even part.

ComplEx (Trouillon et al. 2016) learns a complex diagonal directly, and its asymmetric relations come from imaginary parts. Hayashi & Shimbo (ACL 2017) show the two models are equivalent: any ComplEx model can be converted into a HolE model with the same scores.

The circulant family – the HRR algebra with learned vectors – and a complex diagonal express the same relations, in two bases. Direction lives in the odd part in one, and in the imaginary part in the other.

In-Class Exercise 3

index	0	1	2	3
$\mathbf{u}$	1	2	0	1
$\mathbf{v}$	2	0	1	1

1. Show that DistMult scores (a, r, b) and (b, r, a) the same for every relation vector $\mathbf{r}$.
2. Show that in HolE, swapping the subject and object is the same as replacing $\mathbf{r}$ by its involution: $\mathbf{r} \cdot (\mathbf{b} \# \mathbf{a}) = \mathbf{r}^* \cdot (\mathbf{a} \# \mathbf{b})$. When is a HolE relation symmetric?
3. Take $\mathbf{r} = (0, -2, 0, 2)$. Score the pair $(\mathbf{u}, \mathbf{v})$ in both orders.
4. $\star$ At $d = 32$, RESCAL spends $1{,}024$ numbers per relation and HolE 32. What does RESCAL buy with the other 992, and what does it risk with little data?

Seven minutes. Item 4 is the starred one.

In-Class Exercise 3: Answers

1. $\sum_i a_i\, r_i\, b_i = \sum_i b_i\, r_i\, a_i$: the products commute coordinate by coordinate. A diagonal matrix is symmetric.

2. $(\mathbf{b} \# \mathbf{a})_k = (\mathbf{a} \# \mathbf{b})_{-k}$, so $\mathbf{r} \cdot (\mathbf{b} \# \mathbf{a}) = \sum_k r_k\, (\mathbf{a} \# \mathbf{b})_{-k} = \sum_k r_{-k}\, (\mathbf{a} \# \mathbf{b})_k = \mathbf{r}^* \cdot (\mathbf{a} \# \mathbf{b})$. A HolE relation is symmetric exactly when $\mathbf{r} = \mathbf{r}^*$, that is, when its odd part is zero.

3. $(0, -2, 0, 2) \cdot (3, 4, 3, 6) = -8 + 12 = 4$ in one order, and $(0, -2, 0, 2) \cdot (3, 6, 3, 4) = -12 + 8 = -4$ in the other. A purely odd relation scores the two orders with opposite signs.

4. $\star$ Relations whose matrix is not circulant: any pairing of any coordinate of the subject with any coordinate of the object. With little data those extra numbers fit noise, and they cost 32 times the memory. The bonus deck reports HolE ahead of RESCAL on both standard benchmarks.

Part VI
Attention as a Learned Binding

Attention's Score Is a Learned Form

A head scores a query token against a key token with a learned bilinear form:

$$s(\mathbf{x}, \mathbf{y}) \;=\; (W_Q\mathbf{x}) \cdot (W_K\mathbf{y}) \;=\; \mathbf{x}^\top G\,\mathbf{y}, \qquad G = W_Q^\top W_K, \;\; \operatorname{rank} G \le d_k$$

The course's head, $G = \left(\begin{smallmatrix} 1 & 2 \\ 0 & 1 \end{smallmatrix}\right) = S + A$: $\mathbf{x}_1 = (3,1)$ scores $\mathbf{x}_2 = (1,3)$ at $24 = 16 + 8$, and $\mathbf{x}_2$ scores $\mathbf{x}_1$ at $8 = 16 - 8$. S says how alike; A says which way.

Learning shapes G as on the bilinear-form slides. Every step adds outer products, $(W_K\mathbf{y})\mathbf{x}^\top$ to W_Q and $(W_Q\mathbf{x})\mathbf{y}^\top$ to W_K. Directional data write into A, as in the one-step example; data that reward both directions write into S.

	HRR binding	**attention score**
the map	fixed: the fold	learned: $G = W_Q^\top W_K$
order	invisible; carried by role atoms	visible, through G_A

Saponati et al. (ICML 2025) find more symmetric query–key forms in encoders trained bidirectionally, and more directional ones in autoregressive decoders. Read in this deck's terms, the training data decide what the updates write into A; that reading is ours, not theirs.

Softmax Is a Soft, Learnable Cleanup

Cleanup returns the stored atom with the highest score; attention returns a softmax-weighted average of stored values:

$$\mathrm{cl}(\mathbf{q}) = \arg\max_i \ \mathbf{q} \cdot \mathbf{k}_i, \qquad \mathrm{attn}(\mathbf{q}) = \sum_i \frac{\exp(\mathbf{q} \cdot \mathbf{k}_i/\tau)}{\sum_j \exp(\mathbf{q} \cdot \mathbf{k}_j/\tau)}\, \mathbf{v}_i$$

As the temperature $\tau \to 0$ the softmax puts all its weight on the best key: attention becomes cleanup. The HRR session's query $\mathbf{agt} \# \mathbf{S}$ on *Mark ate the fish*, scored against its nine atoms (mark 0.466; eat and obj 0.035; agt 0.031; the rest 0.006 or less):

temperature τ	1	0.1	0.05	0.02
softmax weight on mark	0.17	0.92	0.999	1.000

Because the softmax is differentiable, the stored keys, which are the cleanup's codebook, can be learned. Ramsauer et al. (ICLR 2021) show that attention's update is the retrieval step of a modern Hopfield network, an associative memory. Attention is a learnable cleanup.

Plate's recurrent networks already cleaned up with a softmax over learned symbol vectors.

Positions: RoPE and HRR's Unitary Powers

HRR marks position m by binding with $\mathbf{p}^m$, for a unitary $\mathbf{p}$: Fourier coefficient k is multiplied by $e^{im\phi_k}$, a rotation by $m\phi_k$. **RoPE** (Su et al. 2024) rotates coordinate pair i of a query or key by $m\theta_i$, with $\theta_i = 10000^{-2i/d}$. Either way, a dot product depends only on the offset:

$$(\mathbf{p}^m \circledast \mathbf{x}) \cdot (\mathbf{p}^{m'} \circledast \mathbf{y}) = \mathbf{x} \cdot (\mathbf{p}^{m'-m} \circledast \mathbf{y}), \qquad (R_m \mathbf{q}) \cdot (R_{m'} \mathbf{k}) = \mathbf{q} \cdot (R_{m'-m}\, \mathbf{k})$$

positions (m, m')	$(0, 3)$	$(5, 8)$	$(10, 13)$	$(2, 7)$
HRR, unitary $\mathbf{p}$, $n = 64$	0.0204	0.0204	0.0204	-0.0006
RoPE, $d = 64$	-12.86	-12.86	-12.86	-14.74

The same idea in two bases. RoPE pairs coordinates and uses designed frequencies; HRR rotates in the Fourier basis, with random phases, or with the shift when $\mathbf{p} = \boldsymbol{\delta}_1$. In the standard recipes neither is learned.

Offsets $3, 3, 3$ give equal scores; offset 5 does not. Session 14 returns to these rotations.

Two Timescales

Both kinds of system learn on two clocks:

	slow: gradients, many examples	**fast: storing, one shot**
transformer	embeddings, W_Q, W_K, W_V	the context, in the KV cache
linear attention	the projections	outer products in the fast weights
classic HRR	nothing	the trace: add a binding
HRR with learned vectors	the atoms, the cleanup	the trace
learned bilinear maps	the form and the embeddings	nothing

In-context learning in a transformer is storing, as in an HRR trace, but exact and growing with the context. Gradient learning in HolE is binding, as in an HRR trace, but weighted by the error.

The two clocks interact: in linear attention and fast-weight memories, the slow network learns *what* to store; the fast memory stores it.

Part VII
Putting It Together

Summary Comparison: How Each System Learns

	classic HRR	**HRR, learned vectors**	**learned bilinear maps**	**transformers**
fixed in advance	atoms, $\circledast$, #	$\circledast$, #	the form's shape	the architecture
learned by gradients	nothing	atoms, cleanup	the form, the embeddings	embeddings, W_Q, W_K, W_V
a new fact gets in by	adding a binding	adding a binding	gradient steps	the context, or gradient steps
examples per fact	one	one; many to train	many	one in context; many to train
a gradient step is	–	an unbinding of the error	an error-weighted outer product	the same, in each head's form
order lives in	role atoms	role atoms, or the odd part of $\mathbf{r}$	G_A; nowhere in DistMult	each head's G_A, and positions
take it apart again?	yes: unbind, clean up	yes	triple by triple	not in general

Two questions decide each column: what is fixed in advance, and where the update can write.

The Research Question, From the Learning Side

An open question for this course: should structure be *installed* in a learned model, or *looked for* in one?

- **Install.** Give a learned model HRR operations: Plate's recurrent networks (1992), HolE (2016), Hrrformer (2023). The structure is guaranteed; learning tunes the vectors and the cleanup.
- **Look.** Probe trained transformers for role-like directions, and measure how much of each head's form is A (final-project card C1).

What this deck adds

A model can learn order only where its updates can write it. DistMult cannot, however long it trains; HolE writes order into the odd part of $\mathbf{r}$; an attention head writes it into the antisymmetric part of $W_Q^\top W_K$. A probe should look where the update rule allows order to be.

And storing is not a weaker kind of learning: a transformer's in-context learning is storing, and HRR's limit there is room, not the rule.

Summary

1. HRR learns by **storing**: a fact gets in with one addition, $\mathbf{t} \leftarrow \mathbf{t} + \mathbf{k} \circledast \mathbf{v}$. It is the Hebbian outer-product rule, folded; its price is interference, about $1/\sqrt{k+1}$ with k pairs, and decay trades the past for recency.
2. A better storing rule cannot fix an n-number trace: the limit is room. The delta rule helps an outer-product memory, which has room.
3. Every binding is an outer product followed by a linear map. HRR fixes the map; bilinear models and attention learn it.
4. **Gradients**: on a bilinear form, each step adds a weighted outer product; through a binding, the gradient is an unbinding. From zero, the first step is the involution decoder and the limit is the exact inverse; weak frequencies learn last, and unitary vectors remove the spread.
5. A model learns order only where its update can write it: not in DistMult, in the odd part of HolE's $\mathbf{r}$, in G_A for RESCAL and attention. Measured, learning put direction exactly where the relation has it.
6. Attention is a learned form plus a soft, learnable cleanup; RoPE and HRR's unitary powers encode position the same way.

Exercises

1. In numpy, implement $\circledast$ and $\#$ with `np.fft`, and confirm by finite differences that the gradient of $\frac{1}{2}\|\mathbf{a} \circledast \mathbf{b} - \mathbf{t}\|^2$ with respect to $\mathbf{a}$ is $\mathbf{b} \# (\mathbf{a} \circledast \mathbf{b} - \mathbf{t})$.
2. Rerun the hand example for 40 steps and plot the error at each frequency. Repeat with $\mathbf{v}$ replaced by a unitary vector, and with $\eta = 1$.
3. Build the family knowledge graph. Train DistMult and HolE, then measure the odd share of each learned HolE relation. Add a relation of your own, symmetric or not, and predict its odd share before training.
4. Store 50 irregular plurals in an HRR trace with decay $\lambda \in \{1, 0.95, 0.9\}$. Plot recall against age, and find the λ that maximizes the number of plurals cleanup still retrieves.
5. Check the relative-position identity for RoPE and for unitary powers. What goes wrong for HRR if $\mathbf{p}$ is not unitary?
6. $\star$ Train a one-head attention layer on a task that needs direction (predict the next token of a b pairs), starting from tied weights, $W_Q = W_K$. Track $\|G_A\|^2 / \|G\|^2$ during training.

Reading, 1: Memories, Storing, and Relations

- T. A. Plate, Holographic recurrent networks, *NIPS 5*, 1992; Holographic reduced representations, *IEEE Transactions on Neural Networks* 6(3), 1995.
- J. Schmidhuber, Learning to control fast-weight memories: an alternative to dynamic recurrent networks, *Neural Computation* 4(1), 1992.
- J. Ba, G. Hinton, V. Mnih, J. Leibo & C. Ionescu, Using fast weights to attend to the recent past, *NeurIPS* 2016.
- I. Schlag, K. Irie & J. Schmidhuber, Linear transformers are secretly fast weight programmers, *ICML* 2021. Linear attention as storing; the delta rule.
- M. Nickel, V. Tresp & H.-P. Kriegel, A three-way model for collective learning on multi-relational data, *ICML* 2011. RESCAL.
- B. Yang, W. Yih, X. He, J. Gao & L. Deng, Embedding entities and relations for learning and inference in knowledge bases, *ICLR* 2015. DistMult.
- M. Nickel, L. Rosasco & T. Poggio, Holographic embeddings of knowledge graphs, *AAAI* 2016. HolE.

Reading, 2: Learned HRRs and Attention

- T. Trouillon, J. Welbl, S. Riedel, É. Gaussier & G. Bouchard, Complex embeddings for simple link prediction, *ICML* 2016. ComplEx.
- K. Hayashi & M. Shimbo, On the equivalence of holographic and complex embeddings for link prediction, *ACL* 2017.
- A. Ganesan, H. Gao, S. Gandhi, E. Raff, T. Oates, J. Holt & M. McLean, Learning with holographic reduced representations, *NeurIPS* 2021. The unitary projection.
- H. Ramsauer et al., Hopfield networks is all you need, *ICLR* 2021. Attention as an associative memory.
- J. Su et al., RoFormer: enhanced transformer with rotary position embedding, *Neurocomputing* 568, 2024.
- M. M. Alam, E. Raff, S. Biderman, T. Oates & J. Holt, Recasting self-attention with holographic reduced representations, *ICML* 2023. Hrrformer.
- M. Saponati et al., The underlying structures of self-attention: symmetry, directionality, and emergent dynamics in Transformer training, *ICML* 2025.